

Meridian — How the merging works

This is the written version of the algorithm in `packages/core`. It is the document to read before changing anything in `text.js`.

Written: 17 September 2026

Covers: weeks 1 to 5 of the work plan

1. The one rule everything follows

> The result must depend only on **which** changes have arrived, never on **when** > they arrived or **in what order**.

Every decision below comes from that rule. Any code that breaks it is a bug, even if the tests happen to pass.

This is why the design never uses:

- the computer's clock — two machines disagree, so the same changes would merge differently on each one;
 - arrival order — the network is allowed to deliver in any order it likes;
 - "who connected first" — that would need a server to decide, and there is no server here.
-

2. Naming changes

Every change gets an id: `"<counter>@<site>"`, for example `"12@a3f9c1d8"`.

- **site** is a random id made once per device.
- **counter** goes up by one for every change that device makes.

Two ids are compared by counter first, then by site. The site part is only a tie-break. It does not matter which device wins a tie — only that every device picks the same winner.

When a device loads a saved document, it moves its counter past every id it has seen. Without that, it would hand out an id it had already used, and two different changes would share one name.

Code: `src/id.js`.

3. Text as a tree, not a string

The document is stored as a tree of single letters. Each letter holds:

FIELD	MEANING
<code>id</code>	Its permanent name
<code>parent</code>	The id of the letter it was typed after, or <code>null</code> for the start
<code>value</code>	One character
<code>deleted</code>	Whether it is hidden

The text you read is the tree walked depth-first: a letter, then everything typed after that letter, then the next sibling.

The tie-break

Several letters can share a parent. That is what a conflict is. They are sorted by `id`, newest first.

Because the tree is built only from `id` and `parent` — both of which never change — and the sort is fixed, every device builds the same tree from the same set of changes. The same tree read the same way gives the same text. No server decides anything.

Why typed words stay whole

When you type `hello`, each letter's parent is the letter before it. The word is a **chain**, not five siblings.

If someone else types `world` in the same spot, their chain hangs off the same parent as your `h`. The sort puts one whole chain before the other:

`helloworld` or `worldhello`

never `hwelorld`. This is the main reason the tree is shaped this way.

Code: `src/text.js`.

4. Deleting

A deleted letter is marked hidden and kept. It is called a tombstone.

It cannot actually be removed, because another device may still send a change that refers to it. If it were gone, that change would have nowhere to attach.

Two things follow:

- **Position mapping matters.** Hidden letters must not be counted when working out where a new letter goes. Getting this wrong looks correct until something is deleted. There is a test for exactly this.
 - **Tombstones pile up.** Removing them safely needs proof that every device has moved past them. That is snapshot work, and it is not built yet.
-

5. Changes that arrive early

A change may arrive before the change it depends on. Two cases:

CASE	WHAT HAPPENS
An insert whose parent is unknown	Held in <code>waiting</code> , keyed by the parent it needs. Applied the moment that parent arrives.
A delete for an unknown letter	Its target is remembered in <code>waitingDeletes</code> . When the letter arrives, it is born hidden.

Both are required, not defensive extras: the convergence test delivers changes in random order, so this path runs constantly.

`text.settled()` reports whether anything is still held back. A replica that has received every change and is not settled means something referred to a letter that was never sent — a real bug, and the tests assert on it.

6. Applying a change twice

Every apply path is written so that applying the same change again does nothing:

- insert — ignored if the id is already known;
- delete — already hidden, so nothing changes;
- field — the same id cannot beat itself;
- tag — adding an id to a set twice is one entry.

This matters because a peer is allowed to resend anything at any time, and after a reconnect it usually will.

7. The other two data types

Fields (`src/map.js`) — one value per name, such as the title. Highest id wins. Losing one of two titles is correct; there is no sensible way to merge them. What matters is that everyone loses the same one.

Tags (`src/set.js`) — a set where adding beats removing. Each add carries its own tag. A remove can only remove tags it could actually see, so a tag added at the same moment as a remove survives.

Add-wins is the safer default for shared documents: losing a tag someone just added is confusing, while seeing one you thought you removed is merely annoying.

8. Syncing

Sending the whole document on every meeting would work and would be unusable on a slow connection. Instead each device keeps a small summary, called a version vector: for each device, the highest counter it holds **with no gaps**.

```
{ "a3f9c1d8": 42, "7b2e0c11": 17 }
```

Two peers swap summaries and each replies with only what the other lacks.

The "no gaps" part is important. If a device holds change 1 and change 3 but not 2, its summary says 1. Claiming 3 would make a peer skip change 2 forever.

A change already held may occasionally be re-sent when it sits past a gap in the other side's history. That is a little extra traffic in exchange for a summary that stays tiny, and applying a change twice does nothing.

Code: `src/oplog.js`.

9. How this is tested

TEST	WHAT IT PROVES
<code>text.test.js</code>	The specific cases that are easy to get wrong, including editing after a deletion and receiving changes backwards
<code>types.test.js</code>	Fields, tags, the version vector, and save/load
<code>convergence.test.js</code>	The real proof: hundreds of random runs where replicas edit apart and then receive everything in different random orders

Every run prints its seed. Replay a failure with:

```
bash SEED=12345 npm test
```

The test suite has teeth

A test that cannot fail is worthless. The sibling sort was deliberately replaced with arrival order — the exact bug the tie-break rule prevents — and 9 tests failed, including all 5 convergence tests. Then it was put back.

Worth repeating after any change to `text.js`.

10. What is known to be missing

Written down honestly, because a limitation you know about is a plan and one you have hidden is a trap.

LIMITATION	EFFECT	WHEN
Interleaving	Two people editing <i>inside</i> the same concurrent run can still mix. Convergence is unaffected — everyone sees the same mixed result. Fixing it means a stronger ordering rule, such as Fugue or YATA.	Later
Tombstones are never dropped	Memory grows with every deletion ever made	Snapshot work
The change log is never trimmed	A snapshot keeps every change, so loading a long-lived document gets slower	Snapshot work
Reading order is rebuilt after each change	Cached, so a burst costs one walk — but it is still $O(\text{document})$ per edit and will need a proper index	Performance week
No encryption yet	Changes travel as plain data	Week 11
No network yet	<code>syncTo</code> works in memory only	Week 7

The first one is the interesting one, and it is a deliberate trade. Convergence was the goal for week 4, and convergence is proven. Better ordering is an improvement on top of something correct, not a missing foundation.